

Práctica 2B – SQL Injection

Seguridad en el Desarrollo de Aplicaciones

Alumno: Luz Celeste López Rivera

Docente: Gerardo Ramírez Villarreal

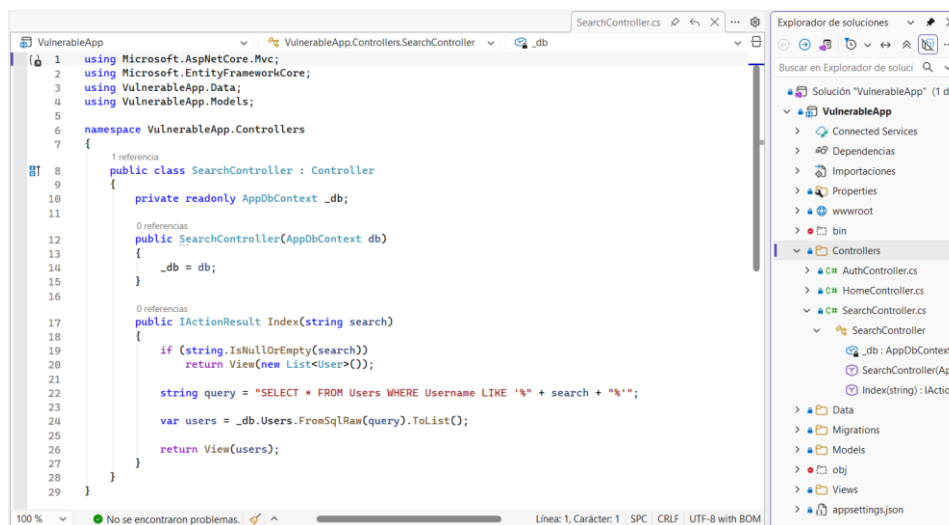
Objetivo

El objetivo de esta práctica fue implementar una funcionalidad vulnerable de búsqueda de usuarios utilizando consultas SQL dinámicas, con el fin de demostrar el funcionamiento de una vulnerabilidad SQL Injection dentro de una aplicación ASP.NET Core MVC.

Desarrollo de la práctica

1. Creación del controlador SearchController

Se creó un controlador llamado SearchController encargado de realizar búsquedas de usuarios dentro de la base de datos.



El controlador recibe el valor ingresado por el usuario y construye una consulta SQL utilizando concatenación directa de cadenas.

2. Implementación de la consulta vulnerable

La consulta vulnerable utilizada fue la siguiente:

```
string query = "SELECT * FROM Users WHERE Username LIKE '%" + search + "%'";
```

Posteriormente la consulta era ejecutada mediante:

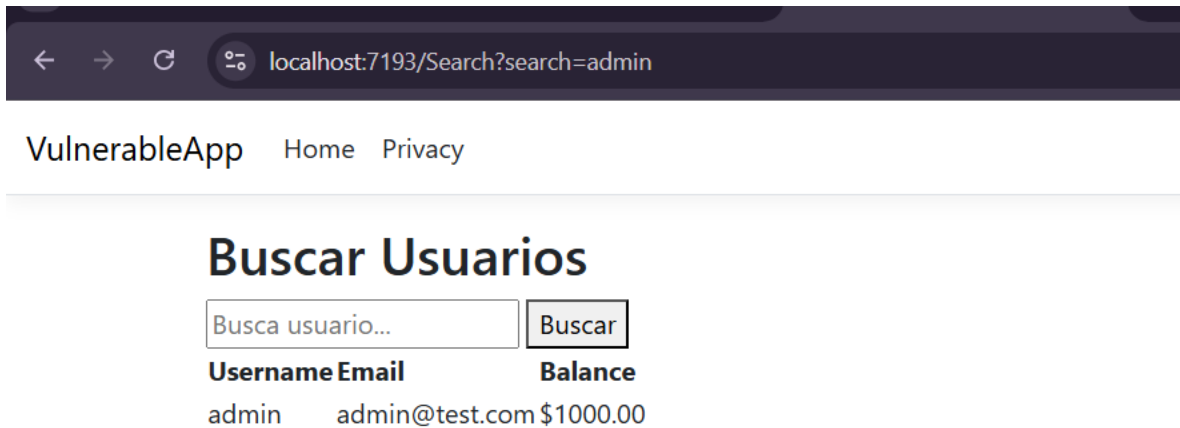
```
var users = _db.Users.FromSqlRaw(query).ToList();
```

La vulnerabilidad ocurre debido a que los datos ingresados por el usuario son concatenados directamente dentro de la consulta SQL sin validaciones ni consultas parametrizadas.

3. Creación de la vista de búsqueda

Se creó la vista Index.cshtml dentro de la carpeta Views/Search.

- Campo de búsqueda
- Botón de búsqueda
- Tabla de resultados



Pruebas realizadas

Búsqueda legítima

Se realizaron búsquedas normales utilizando:

- admin
- user1

El sistema mostró únicamente los registros correspondientes.

Prueba de SQL Injection

Posteriormente se utilizó el siguiente payload:

```
' OR '1'='1
```

Al ejecutar la búsqueda, la aplicación devolvió todos los registros de la tabla Users.

Esto confirmó exitosamente la existencia de la vulnerabilidad SQL Injection.

Explicación técnica de la vulnerabilidad

La vulnerabilidad corresponde a SQL Injection (CWE-89 / OWASP Injection).

El problema pasa porque la consulta SQL es construida mediante concatenación directa de cadenas utilizando datos controlados por el usuario.

Al ingresar:

```
' OR '1'='1
```

la condición lógica de la consulta se altera y provoca que siempre sea verdadera.

Esto permite:

- Acceso no autorizado a información
- Exposición de registros
- Manipulación de consultas SQL
- Posible evasión de controles de seguridad

Resultados obtenidos

La aplicación vulnerable permitió:

- Ejecutar búsquedas normales
- Ejecutar búsquedas manipuladas
- Exponer todos los registros de la tabla Users
- Demostrar exitosamente SQL Injection

Conclusión

La práctica permitió comprender el funcionamiento de una vulnerabilidad SQL Injection dentro de aplicaciones web que utilizan consultas dinámicas sin parametrización.

También permitió observar cómo datos controlados por el usuario pueden alterar completamente la lógica de una consulta SQL y provocar exposición de información sensible.